

module-13-pdf-word

Chapter 13: PDF and Word Documents | 23 programs in learning order

Source: *Automate the Boring Stuff with Python 1e*, Al Sweigart (CC-BY-NC-SA). Syllabus Ch13: PyPDF2 extract/decrypt/combine, python-docx read/write

How to use this guide (beginner path)

1) Read Concepts once. 2) Study the Visuals - redraw each on paper. 3) Follow the Program map. 4) Work programs 01..N in order. [shell]: type every plain line into IDLE, compare with # OUT: comments (errors shown are intentional demos). [script]: save, predict output, run with F5. 5) Drill Interview prep. 6) Teach it: explain each visual aloud without looking.

Concepts in detail

PDFs: PdfReader pages give `extract_text()`; `decrypt('pw')` unlocks; PdfWriter `add_page()/write()` merges - the combiner project loops a folder of PDFs into one. Word docs: `Document('f.docx').paragraphs` lists text; `getText()` joins it; runs carry bold/italic/underline flags; `add_paragraph()/add_heading()/add_break()/add_picture()` author new files - the invitations project mail-merges names into documents.

Key ideas to carry forward

- Reader pages are 0-indexed: `reader.pages[0].extract_text()`.
- Runs, not paragraphs, hold formatting - iterate `paragraph.runs` for styles.
- Encrypted PDFs must `decrypt()` before any page reads.

Classic beginner mistakes

- ! Assuming `extract_text()` preserves layout perfectly (it approximates).
- ! Forgetting `add_picture` sizing so images explode the page.

Study order hint: 01-06 PDF read/decrypt/create, 07-10 combiner, 11-16 Word read/runs, 17-23 Word authoring + invitations.

Interview prep - questions you must answer cold

Say each answer aloud in under 60 seconds, then prove it with one program from this module.

Q1. Extract text from a PDF?

A: PdfReader(path).pages[i].extract_text() - pages are 0-indexed. Decrypt first if encrypted.

Q2. Merge many PDFs?

A: Loop files, PdfReader each, writer.add_page(p) for every page, writer.write(out) - the combiner capstone.

Q3. Read a .docx?

A: Document(path).paragraphs -> .text each; getText() joins all. Styles live on runs, not paragraphs.

Q4. What is a run?

A: A same-formatted text span inside a paragraph. Iterate paragraph.runs for bold/italic/underline flags.

Q5. Create Word docs in code?

A: add_heading(), add_paragraph(), add_break(), add_picture(path, width=...) - the invitations project mail-merges these.

Q6. PDF text limits?

A: extract_text() approximates layout; scanned PDFs are images (need OCR); tables often need dedicated tools.

Teach it to others

The one analogy that sticks

A PDF is a printed poster (fixed, hard to edit - you overlay/merge pages); a Word doc is a Lego build (paragraph blocks made of run bricks, each brick with its own color = formatting).

What your students will get wrong

- x PDF text keeps perfect layout (approximation only).
- x Paragraphs hold formatting (runs do).
- x `add_page` shares objects safely across readers (copy into one writer).

5-minute whiteboard plan

1. Extract page 1 of a sample PDF live.
2. Merge two PDFs on screen.
3. Inspect runs of a bold sentence.
4. Exit ticket: which call adds an image? (`add_picture`).

Visualize it - draw these from memory

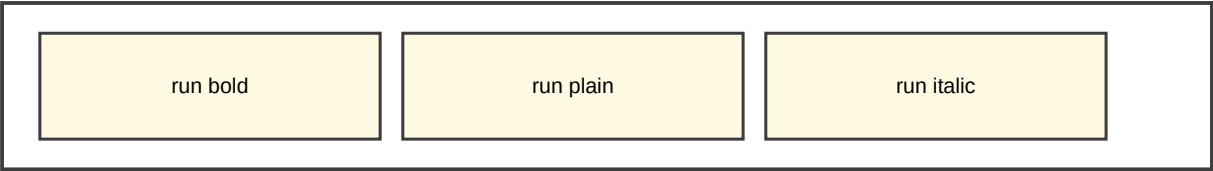
Redraw each figure on paper with labels before you touch code. If you cannot draw it, you cannot teach it.

Visual 1 - merger funnel: many PDFs, one writer



Reader per file, add_page every page, single write. Decrypt before reading encrypted files.

Visual 2 - a paragraph is built of run bricks



Formatting lives on runs, not paragraphs. Iterate paragraph.runs to restyle precisely.

More examples - same idea, new clothes

Three bonus drills per module. Cover the output, predict, then run.

X1. PDF page counter

```
from pypdf import PdfReader
r = PdfReader('a.pdf')
print(len(r.pages), 'pages')
print(bool(r.is_encrypted))
# EXPECTED: e.g. 12 pages / False
```

X2. Docx word count

```
import docx
d = docx.Document('essay.docx')
words = sum(len(p.text.split()) for p in d.paragraphs)
print(words, 'words')
# EXPECTED: e.g. 850 words
```

X3. Certificate maker

```
import docx
names = ['Al', 'Bo']
d = docx.Document()
for n in names:
    d.add_heading('Certificate', 0)
    d.add_paragraph('Awarded to ' + n)
    d.add_page_break()
d.save('certs.docx')
print('saved')
# EXPECTED: certs.docx with 2 pages
```

Program map - all 23 at a glance

```
01 01_extracting_text_from.py
02 02_decrypting_pdfs.py
03 03_creating_pdfs.py
04 04_creating_pdfs_pypdf2.py
05 05_creating_pdfs_pypdf2_2.py
06 06_creating_pdfs_pypdf2_3.py
07 07_step_1_find.py
08 08_step_2_open.py
09 09_step_3_add.py
10 10_step_4_save.py
11 11_reading_word_documents.py
12 12_getting_full_text.py
13 13_getting_full_text_readdocx.py
14 14_getting_full_text_fulltext.py
15 15_getting_full_text_2.py
16 16_run_attributes.py
17 17_writing_word_documents.py
18 18_writing_word_documents_docx.py
19 19_writing_word_documents_2.py
20 20_adding_headings.py
21 21_adding_line_page.py
22 22_adding_pictures.py
23 23_custom_invitations_as.py
```

Topic: Extracting Text from PDFs

01/23 01_extracting_text_from.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import PyPDF2
pdfFileObj = open('meetingminutes.pdf', 'rb')
pdfReader = PyPDF2.PdfFileReader(pdfFileObj)
pdfReader.numPages
# OUT: 19
pageObj = pdfReader.getPage(0)
pageObj.extractText()
# OUT: 'OFFFFFIICCIIAALL BB00AARRDD MMIINNUUTTEESS Meeting of March 7, 2015
# OUT: \n      The Board of Elementary and Secondary Education shall provide leadership
# OUT: and create policies for education that expand opportunities for children,
# OUT: empower families and communities, and advance Louisiana in an increasingly
# OUT: competitive global market. BOARD of ELEMENTARY and SECONDARY EDUCATION '
```

Topic: Decrypting PDFs

02/23 02_decrypting_pdfs.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import PyPDF2
pdfReader = PyPDF2.PdfFileReader(open('encrypted.pdf', 'rb'))
pdfReader.isEncrypted
# OUT: True
pdfReader.getPage(0)
# OUT: Traceback (most recent call last):
# OUT:   File "<pyshell#173>", line 1, in <module>
# OUT:     pdfReader.getPage()
# OUT:   --snip--
# OUT:   File "C:\Python34\lib\site-packages\PyPDF2\pdf.py", line 1173, in getObject
# OUT:     raise utils.PdfReadError("file has not been decrypted")
# OUT: PyPDF2.utils.PdfReadError: file has not been decrypted
pdfReader.decrypt('rosebud')
# OUT: 1
pageObj = pdfReader.getPage(0)
```

Topic: Creating PDFs

03/23 03_creating_pdfs.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import PyPDF2
pdf1File = open('meetingminutes.pdf', 'rb')
pdf2File = open('meetingminutes2.pdf', 'rb')
pdf1Reader = PyPDF2.PdfFileReader(pdf1File)
pdf2Reader = PyPDF2.PdfFileReader(pdf2File)
pdfWriter = PyPDF2.PdfFileWriter()

for pageNum in range(pdf1Reader.numPages):
    # OUT:         pageObj = pdf1Reader.getPage(pageNum)
    # OUT:         pdfWriter.addPage(pageObj)

for pageNum in range(pdf2Reader.numPages):
    # OUT:         pageObj = pdf2Reader.getPage(pageNum)
    # OUT:         pdfWriter.addPage(pageObj)

pdfOutputFile = open('combinedminutes.pdf', 'wb')
pdfWriter.write(pdfOutputFile)
pdfOutputFile.close()
pdf1File.close()
pdf2File.close()
```

04/23 04_creating_pdfs_pypdf2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
import PyPDF2  
minutesFile = open('meetingminutes.pdf', 'rb')  
pdfReader = PyPDF2.PdfFileReader(minutesFile)  
page = pdfReader.getPage(0)  
page.rotateClockwise(90)  
# OUT:    {'/Contents': [IndirectObject(961, 0), IndirectObject(962, 0),  
# OUT:    --snip--  
# OUT:    ]  
pdfWriter = PyPDF2.PdfFileWriter()  
pdfWriter.addPage(page)  
resultPdfFile = open('rotatedPage.pdf', 'wb')  
pdfWriter.write(resultPdfFile)  
resultPdfFile.close()  
minutesFile.close()
```

05/23 05_creating_pdfs_pypdf2_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
import PyPDF2  
minutesFile = open('meetingminutes.pdf', 'rb')  
pdfReader = PyPDF2.PdfFileReader(minutesFile)  
minutesFirstPage = pdfReader.getPage(0)  
pdfWatermarkReader = PyPDF2.PdfFileReader(open('watermark.pdf', 'rb'))  
minutesFirstPage.mergePage(pdfWatermarkReader.getPage(0))  
pdfWriter = PyPDF2.PdfFileWriter()  
pdfWriter.addPage(minutesFirstPage)  
  
for pageNum in range(1, pdfReader.numPages):  
# OUT:          pageObj = pdfReader.getPage(pageNum)  
# OUT:          pdfWriter.addPage(pageObj)  
resultPdfFile = open('watermarkedCover.pdf', 'wb')  
pdfWriter.write(resultPdfFile)  
minutesFile.close()  
resultPdfFile.close()
```

06/23 06_creating_pdfs_pypdf2_3.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
import PyPDF2  
pdfFile = open('meetingminutes.pdf', 'rb')  
pdfReader = PyPDF2.PdfFileReader(pdfFile)  
pdfWriter = PyPDF2.PdfFileWriter()  
for pageNum in range(pdfReader.numPages):  
# OUT:         pdfWriter.addPage(pdfReader.getPage(pageNum))  
  
pdfWriter.encrypt('swordfish')  
resultPdf = open('encryptedminutes.pdf', 'wb')  
pdfWriter.write(resultPdf)  
resultPdf.close()
```

Topic: Step 1: Find All PDF Files

07/23 07_step_1_find.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# combinePdfs.py - Combines all the PDFs in the current working directory into
# into a single PDF.

import PyPDF2, os

# Get all the PDF filenames.
pdfFiles = []
for filename in os.listdir('.'):
    if filename.endswith('.pdf'):
        pdfFiles.append(filename)
pdfFiles.sort(key=str.lower)

pdfWriter = PyPDF2.PdfFileWriter()

# TODO: Loop through all the PDF files.

# TODO: Loop through all the pages (except the first) and add them.

# TODO: Save the resulting PDF to a file.
```

Topic: Step 2: Open Each PDF

08/23 08_step_2_open.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# combinePdfs.py - Combines all the PDFs in the current working directory into
# a single PDF.

import PyPDF2, os

# Get all the PDF filenames.
pdfFiles = []
--snip--
# Loop through all the PDF files.
for filename in pdfFiles:
    pdfFileObj = open(filename, 'rb')
    pdfReader = PyPDF2.PdfFileReader(pdfFileObj)
    # TODO: Loop through all the pages (except the first) and add them.

# TODO: Save the resulting PDF to a file.
```

Topic: Step 3: Add Each Page

09/23 09_step_3_add.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# combinePdfs.py - Combines all the PDFs in the current working directory into
# a single PDF.

import PyPDF2, os

--snip--

# Loop through all the PDF files.
for filename in pdfFiles:
    --snip--
    # Loop through all the pages (except the first) and add them.
    for pageNum in range(1, pdfReader.numPages):
        pageObj = pdfReader.getPage(pageNum)
        pdfWriter.addPage(pageObj)

# TODO: Save the resulting PDF to a file.
```

Topic: Step 4: Save the Results

10/23 10_step_4_save.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# combinePdfs.py - Combines all the PDFs in the current working directory into
# a single PDF.
import PyPDF2, os

--snip--

# Loop through all the PDF files.
for filename in pdfFiles:
    --snip--
    # Loop through all the pages (except the first) and add them.
    for pageNum in range(1, pdfReader.numPages):
        --snip--

# Save the resulting PDF to a file.
pdfOutput = open('allminutes.pdf', 'wb')
pdfWriter.write(pdfOutput)
pdfOutput.close()
```

Topic: Reading Word Documents

11/23 11_reading_word_documents.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import docx
doc = docx.Document('demo.docx')
len(doc.paragraphs)
# OUT:      7
doc.paragraphs[0].text
# OUT:      'Document Title'
doc.paragraphs[1].text
# OUT:      'A plain paragraph with some bold and some italic'
len(doc.paragraphs[1].runs)
# OUT:      4
doc.paragraphs[1].runs[0].text
# OUT:      'A plain paragraph with some '
doc.paragraphs[1].runs[1].text
# OUT:      'bold'
doc.paragraphs[1].runs[2].text
# OUT:      ' and some '
# OUT: ? >>> doc.paragraphs[1].runs[3].text
# OUT:      'italic'
```

Topic: Getting the Full Text from a .docx File

12/23 12_getting_full_text.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3

import docx

def getText(filename):
    doc = docx.Document(filename)
    fullText = []
    for para in doc.paragraphs:
        fullText.append(para.text)
    return '\n'.join(fullText)
```

13/23 13_getting_full_text_readdocx.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
import readDocx  
print(readDocx.getText('demo.docx'))  
# OUT: Document Title  
# OUT: A plain paragraph with some bold and some italic  
# OUT: Heading, level 1  
# OUT: Intense quote  
# OUT: first item in unordered list  
# OUT: first item in ordered list
```

14/23 14_getting_full_text_fulltext.py [script]

Script: save and run; predict the output first, then compare.

```
fullText.append(' ' + para.text)
```

15/23 15_getting_full_text_2.py [script]

Script: save and run; predict the output first, then compare.

```
return '\n\n'.join(fullText)
```

Topic: Run Attributes

16/23 16_run_attributes.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.
Lines starting with >>> are input; other lines are expected output (commented).

```
doc = docx.Document('demo.docx')
doc.paragraphs[0].text
# OUT: 'Document Title'
doc.paragraphs[0].style
# OUT: 'Title'
doc.paragraphs[0].style = 'Normal'
doc.paragraphs[1].text
# OUT: 'A plain paragraph with some bold and some italic'
(doc.paragraphs[1].runs[0].text, doc.paragraphs[1].runs[1].text, doc.
# OUT: paragraphs[1].runs[2].text, doc.paragraphs[1].runs[3].text)
# OUT: ('A plain paragraph with some ', 'bold', ' and some ', 'italic')
doc.paragraphs[1].runs[0].style = 'QuoteChar'
doc.paragraphs[1].runs[1].underline = True
doc.paragraphs[1].runs[3].underline = True
doc.save('restyled.docx')
```

Topic: Writing Word Documents

17/23 17_writing_word_documents.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import docx
doc = docx.Document()
doc.add_paragraph('Hello world!')
# OUT: <docx.text.Paragraph object at 0x0000000003B56F60>
doc.save('helloworld.docx')
```

18/23 18_writing_word_documents_docx.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
import docx  
doc = docx.Document()  
doc.add_paragraph('Hello world!')  
# OUT: <docx.text.Paragraph object at 0x000000000366AD30>  
paraObj1 = doc.add_paragraph('This is a second paragraph.')  
paraObj2 = doc.add_paragraph('This is a yet another paragraph.')  
paraObj1.add_run(' This text is being added to the second paragraph.')  
# OUT: <docx.text.Run object at 0x0000000003A2C860>  
doc.save('multipleParagraphs.docx')
```

19/23 19_writing_word_documents_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
```

```
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
doc.add_paragraph('Hello world!', 'Title')
```

Topic: Adding Headings

20/23 20_adding_headings.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
doc = docx.Document()
doc.add_heading('Header 0', 0)
# OUT: <docx.text.Paragraph object at 0x00000000036CB3C8>
doc.add_heading('Header 1', 1)
# OUT: <docx.text.Paragraph object at 0x00000000036CB630>
doc.add_heading('Header 2', 2)
# OUT: <docx.text.Paragraph object at 0x00000000036CB828>
doc.add_heading('Header 3', 3)
# OUT: <docx.text.Paragraph object at 0x00000000036CB2E8>
doc.add_heading('Header 4', 4)
# OUT: <docx.text.Paragraph object at 0x00000000036CB3C8>
doc.save('headings.docx')
```

Topic: Adding Line and Page Breaks

21/23 21_adding_line_page.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
doc = docx.Document()
doc.add_paragraph('This is on the first page!')
# OUT:    <docx.text.Paragraph object at 0x0000000003785518>
doc.paragraphs[0].runs[0].add_break(docx.text.WD_BREAK.PAGE)
doc.add_paragraph('This is on the second page!')
# OUT:    <docx.text.Paragraph object at 0x00000000037855F8>
doc.save('twoPage.docx')
```

Topic: Adding Pictures

22/23 22_adding_pictures.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
```

```
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
doc.add_picture('zophie.png', width=docx.shared.Inches(1),
```

```
# OUT: height=docx.shared.Cm(4))
```

```
# OUT: <docx.shape.InlineShape object at 0x00000000036C7D30>
```

Topic: Custom Invitations as Word Documents

23/23 23_custom_invitations_as.py [script]

Script: save and run; predict the output first, then compare.

Prof. Plum
Miss Scarlet
Col. Mustard
Al Sweigart
Robocop